

Capstone Project Submission

Grade: F

Status: Automatic Fail from LLM

Feedback

Score Summary Table

- Data Pipeline -- 13/15 -- Weather ingestion + embeddings job is idempotent and writes to Lakebase via UPSERT; job YAML provided, but the notebook doesn't use Spark and pipeline orchestration evidence is light.
- Third-Party API Integration -- 15/15 -- Working integrations with NWS (api.weather.gov), Open?Meteo geocoding/forecast; robust error handling (timeouts, raise_for_status); data is persisted and used downstream.
- Unstructured Data Processing -- 15/15 -- Narrative text is chunked and embedded with sentence-transformers; embeddings stored in pgvector with HNSW; semantic search is parameterized and exercised.
- Databricks App with Frontend -- 8/15 -- Databricks Apps for dashboard and MCP are running; Flask API exposes weather sync/search. Frontend watchlist UI routes don't match backend endpoints, and there's no weather UI, so core flows are partially broken/only via API.
- AI Agent with Tools (Read + Write Actions) -- 17/30 -- Read tools: 10/10 (get_current_weather, get_forecast, predict_umbrella_needed used via Agent Bricks endpoint). Write tools: 0/10 (no tools that mutate Lakebase). Agent quality: 7/10 (good validation, clear prompt, tool use evidenced; limited scope).
- Databricks App with Frontend -- 8/15 -- Already counted above. [Note: keeping one row per rubric component; total computed below.]

Total Score

- Raw subtotal: 68/100
- Capped total: 60/100 (cap applied because the Agent is read-only with no write actions)

Strengths

- Solid, reproducible weather pipeline: normalized NWS documents -> text chunking -> pgvector embeddings with HNSW; idempotent UPSERTs in weather_documents and weather_embeddings.
- Correct, safe API usage with input validation, timeouts, raise_for_status, and HTTP error mapping; test coverage for both service and API layers.
- Clean semantic search implementation using parameterized cosine distance and no string injection; verified by unit test.
- Databricks Apps deployed and reported running; Lakebase vector tables created with correct dimensionality (384) and populated.

Gaps & Deductions

- Agent write actions missing (?10/30 in category 5; cap to 60 overall): The weather MCP exposes only read tools; there is no tool that mutates Lakebase (e.g., save a note, add/remove from watchlist, persist analysis).
- Frontend usability gaps (?7/15 in category 4): index.html calls /watchlist and /watchlist/<symbol> but Flask only implements /api/watchlist (GET only). Watchlist add/remove flows would 404; no weather search UI.
- Pipeline tech mismatch vs claims (?2/15 in category 1): The provided embedding

notebook (ingest_weather_embeddings.py) runs pure Python/psycopg2, not Spark as claimed in parts of the README. A Databricks Job exists, but there's no evidence of Spark-based distributed processing in the submitted weather pipeline.

- Limited end-to-end app UX evidence (?? within category 4): Running status is shown, but there are no authenticated screenshots or a demo of the UI performing the core flows (sync, search, results rendering).

Evidence Gaps

- Massive/Stocks track: massive_client.py and the stock "alpaca_mcp_server.py" with write tools are referenced in README_CAPSTONE.md but not included here. Without those files and a demo, I can't verify the stock project's API usage, embeddings pipeline, or agent write tools.
- Screenshots or a short screencast of the Databricks App frontend performing weather sync/search and showing results. You provided CLI "apps get" output, but not UI evidence.
- Job run logs or a run URL for the Ingest Weather Embeddings job to confirm production execution at scale.
- Clarification on multiple bundle files: there are two databricks.yml/app.yaml sets (weather MCP and "day-3" lakebase app). A brief note on which is the authoritative capstone target would help.
- The "notebooks/ingest_ticker_news_embeddings.py" (Spark) file mentioned in README_CAPSTONE.md is not included; cannot verify any Spark pipeline for the stock project.

Suggestions for Improvement

- Implement at least one agent write tool wired to Lakebase and expose it in Agent Bricks:
 - Example: save_research_note(symbol, title, content) writing to research_notes (05_setup_research_tables.sql is already present).
 - Example: add_to_watchlist(symbol) and remove_from_watchlist(symbol) that upsert/delete rows keyed by user email, so the agent can take actions.
- Provide an Agent Bricks transcript or screenshot showing the tool calls and the resulting Lakebase mutations (before/after query).
- Fix the frontend routes and add a minimal weather UI:
- Align index.html with Flask: use /api/watchlist and implement POST /api/watchlist and DELETE /api/watchlist/<symbol> in app.py or update the JS to call existing endpoints.
- Add a simple page for /weather/search that renders top?k results and sources; include an authenticated screenshot.
- Strengthen pipeline evidence:
 - Attach a Databricks Job run link/logs for the embeddings job, and optionally refactor to Spark UDF or Pandas UDF for batch embedding if volume grows, to align with the "Spark pipeline" claim.
 - Add a short note on idempotency checks and runbook (what to re-run when, expected counts).
- Unify the submission scope:
 - If the official capstone is the weather assistant, prune stock-specific claims from README_CAPSTONE.md to avoid confusion.
 - If the official capstone is the stock assistant, include the missing Massive client code, Spark notebook, schema DDL, and MCP server with write tools, plus an end-to-end demo.

If I've made mistakes in the setup of this request

- If there are additional files (e.g., massive_client.py, mcp_server/alpaca_mcp_server.py, notebooks/ingest_ticker_news_embeddings.py) that demonstrate write tools and a Spark pipeline, please provide them or a repo link and I will re-evaluate.
- If the dashboard UI has an updated index.html or alternative routes (e.g., /api/watchlist POST/DELETE) in a different branch, please share the commit or screenshots.
- If the embeddings job was run in your workspace, a job run URL or exported run logs would let me validate category 1 more strongly.
- If the stock app/agent is the primary capstone, please clarify the target project and provide agent transcripts showing write actions (adding to watchlist, saving notes) with Lakebase row diffs.